

SOFTWARE DESIGN AND ENGINEERING

Course	B.Tech.-III-Sem.	L	T	P	C
Course Code	25CSPC34	3	-	-	3

Course Outcomes (COs)

COs	Upon completion of course the students will be able to	SDGs
CO1	apply software engineering process models and requirements engineering	4,9
CO2	design software systems using architectural and component-level design	4,9
CO3	model software systems using UML diagrams	4,9
CO4	apply testing strategies and quality assurance techniques	4,9
CO5	analyze risks and design scalable systems using modern principles	4,9,12

Syllabus

Unit	Title/Topics	Hours
I	Introduction and Requirements Engineering	8
Introduction to Software Engineering, evolving role of software, SDLC models: Waterfall, Incremental, Agile (overview), software engineering as a layered technology, Capability Maturity Model Integration (CMMI) – overview, functional and non-functional requirements, requirements engineering process: feasibility study, requirements elicitation & analysis, validation & management, software requirements specification (SRS). <i>Task: Document functional and non-functional requirements for online bookstore system.</i>		
II	Design Engineering & System Design (LLD + HLD)	8
Design process and design quality; design concepts: abstraction, modularity, cohesion, coupling; architectural design (HLD): software architecture, architectural styles (layered, client-server, microservices (overview)). Scalability concepts: load balancing, caching. component-level design (LLD): class design, API design basics. user interface design: golden rules, UI/UX basics. <i>Task: Design high-level architecture for an online bookstore using layered architecture.</i>		
III	UML Modeling and Design Visualization	4+4=8
Part-A: Structural Modeling: Principles of modeling, UML overview. Class, object, component diagrams (overview). <i>Task: Design a class diagram for ATM banking system / library management system</i>		
Part-B: Behavioral Modeling: Use case, Sequence, Activity, State chart, Deployment diagrams (overview). <i>Task: Draw use case diagram for ATM operations, sequence diagram for transaction flow.</i>		
IV	Testing & Quality Assurance	8
Software testing strategies, black-box testing & white-box testing, unit testing, integration testing, system testing, validation testing and debugging, software quality metrics, product and process metrics, introduction to automated testing (Selenium/PyTest). <i>Task: Write unit test cases for login module, transaction/payment module.</i>		
V	Software Management & System Design Case Studies	8
Risk management: risk identification, analysis, mitigation, RMMM plan, software quality assurance (SQA), software reviews and reliability, ISO 9000 quality standards (overview), basic system design case studies: URL shortener, e-commerce, chat system (overview). <i>Task: Identify and mitigate risks in e-commerce system design project.</i>		
Textbooks		
1. Roger S. Pressman, Software engineering - A practitioner's Approach, TMH (I), 8 th Edn, 2015. 2. Ian Sommerville, Software Engineering, Pearson education Asia, 10 th Edn, 2015. 3. Grady Booch et al., The Unified Modeling Language User Guide, 2nd Ed., Pearson, 2005.		

References
1. Designing Data-Intensive Applications - Martin Kleppmann, 2017, O'Reilly Media. 2. Clean Architecture - Robert C. Martin, 2017, Prentice Hall (Pearson). 3. Head First Design Patterns - Eric Freeman and Elisabeth Robson.